COSC419/519:DEEP LEARNING

The Attention Mechanism

Dr. Mohamed Shehata

Computer Science

University of British Columbia

OKANAGAN

Building Intuition

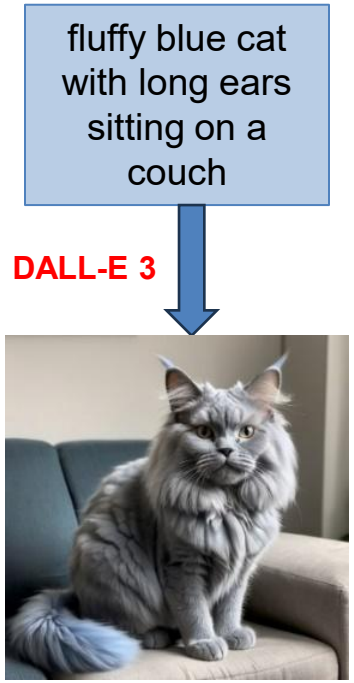
- What is the meaning of the word “crane”?
  - Hard to say the exact meaning without looking at the **context**
- Example:
  - This bird is a crane.

  - They needed a crane to lift the object.

- To understand the meaning, we need the context.
  - That is, we must **attend to** (or pay attention to) the other words in the sentence.
- Key idea of attention: new context vector at every time step. Each context vector will attend to different input regions.
- This is how we interpret words as humans.

Where We Use Attention

- The idea originated from the field of Natural Language Processing, but is now also used for many other applications including Computer Vision.
- A very famous use of the attention mechanism is in ChatGPT.
  - GPT stands for Generative Pre-trained Transformer.
  - Transformers are a relatively new architecture of NN, and the attention mechanism is a crucial part of the architecture.
- In general, its usages could include:
  - Generating text (like ChatGPT)
  - Inscribing voice to text
  - Text-to-image
  - Computer Vision tasks like image generation



31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Citations: 169,248

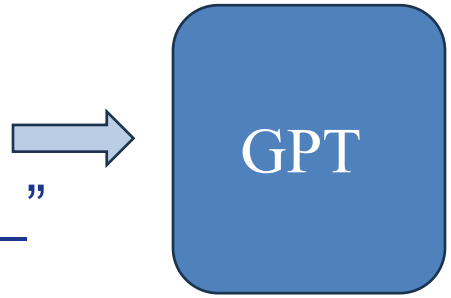
Next Word Prediction

- In NLP, you can give some seed text and ask the model to predict the next word. For example:

Seed Text: “The explorers ventured deep into the uncharted jungle, guided only by the faint traces of a centuries-old map. The air was thick with humidity, and every step seemed to whisper secrets of a forgotten world. Suddenly, one of them stopped and pointed”

Model Prediction: “ahead . Through the dense foliage, a massive stone structure emerged, covered in vines and moss. It was a pyramid, unlike any they had seen before, with intricate carvings that seemed to glow faintly in the dim light. The group approached cautiously, their hearts pounding with a mix of excitement and fear. As they stepped closer, a low hum filled the air, and the ground beneath them began to tremble."

- Predictions might look like:
  - “Suddenly, one of them
  - stopped and pointed ___”



Probability of being the next word

and: 1%
..
ahead: 43%
..
be: 0.2%
..
..
zoo: 0.001%

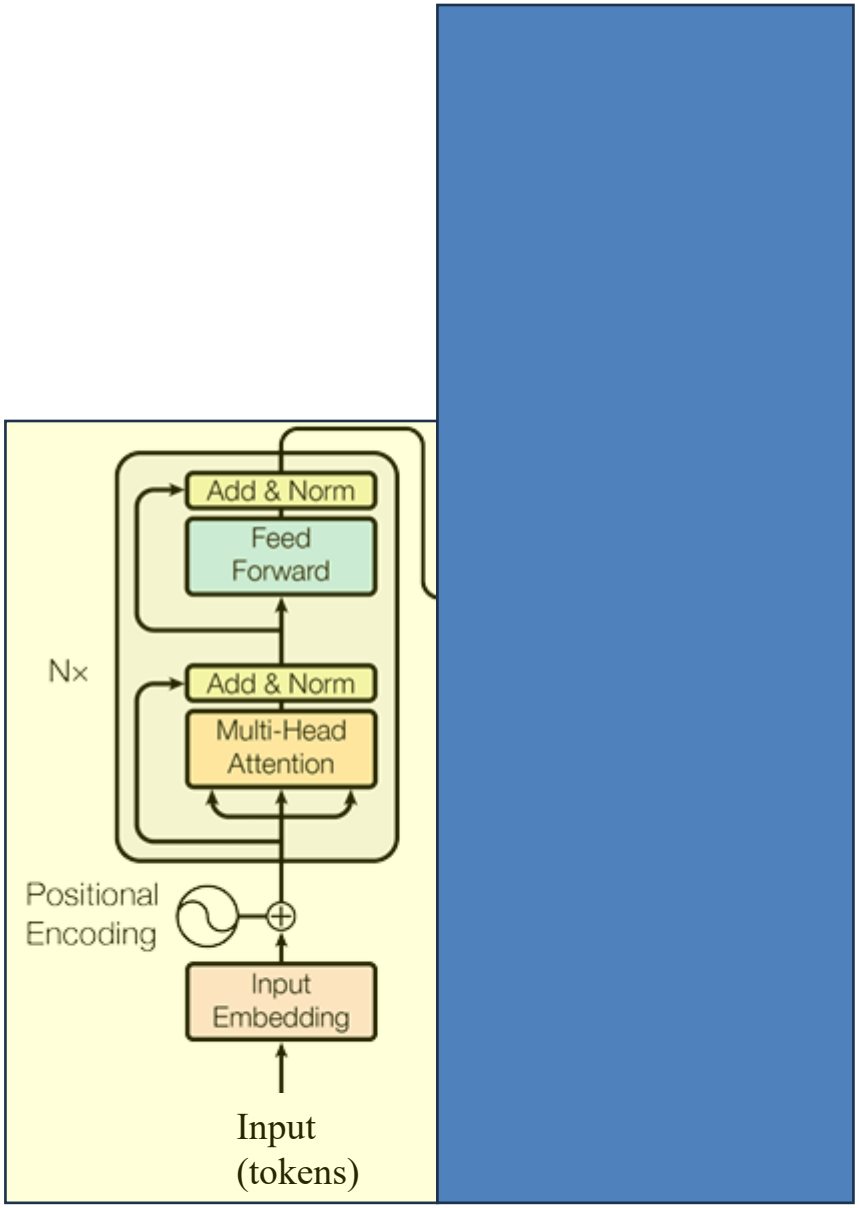


Figure 1: The Transformer - model architecture.

Input Token

- How does a computer see words?

Suddenly, one of them stopped and pointed _____

Tokenize

Suddenly , one of them stopped and pointed _____

$$\begin{bmatrix} 1.2 \\ -0.3 \\ \vdots \\ \vdots \\ 0.7 \end{bmatrix} \quad \begin{bmatrix} -0.3 \\ 2.1 \\ \vdots \\ \vdots \\ 1.9 \end{bmatrix} \quad \cdots \quad \cdots \quad \cdots \quad \cdots \quad \begin{bmatrix} \vdots \\ \vdots \\ \vdots \\ \vdots \\ 0.2 \end{bmatrix} \quad \begin{bmatrix} 2.4 \\ 0.1 \\ \vdots \\ \vdots \\ -0.4 \end{bmatrix}$$

Each token is represented as a vector, called the **embeddings**

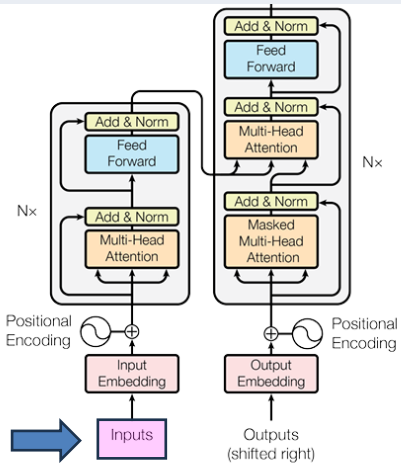


Figure 1: The Transformer - model architecture.

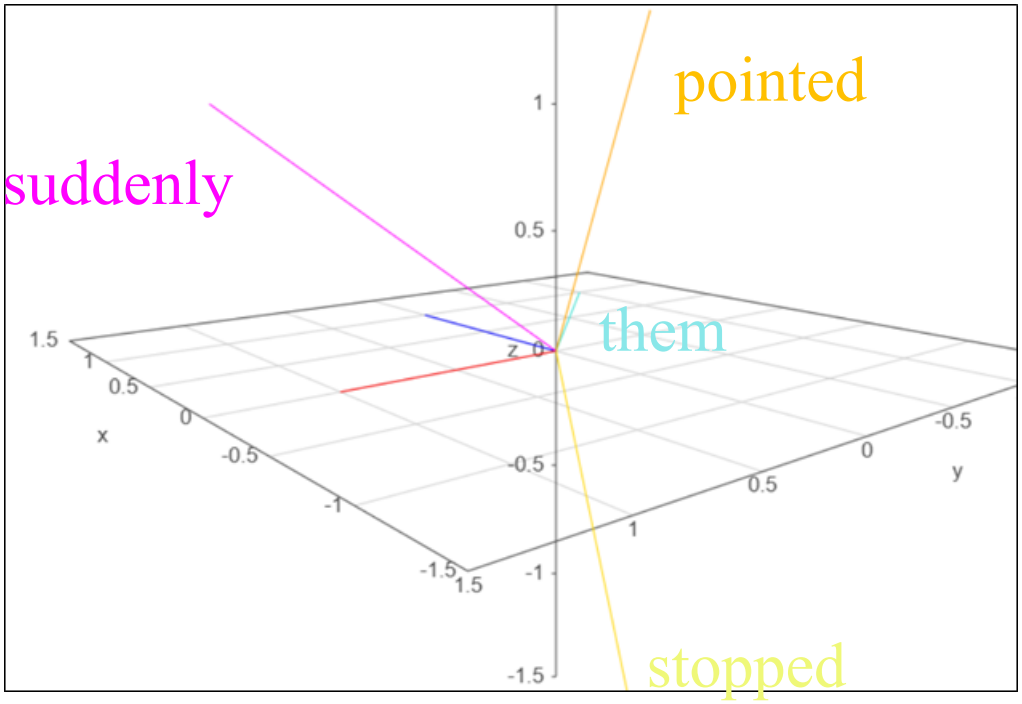
- Each token is mapped to a **dense vector representation** (embedding) using a learned weight matrix, called the **embedding matrix**.
- The **size of the embedding matrix** in GPT3 (for the largest model) is:

vocab_size x d_model =50,257x12,288

What is an Embedding?

- Since embeddings are just vectors of fixed size (say of size D), you can imagine them as points in D-dimensional space!

- Here's a sample visualization of 3-dimensional word embeddings:

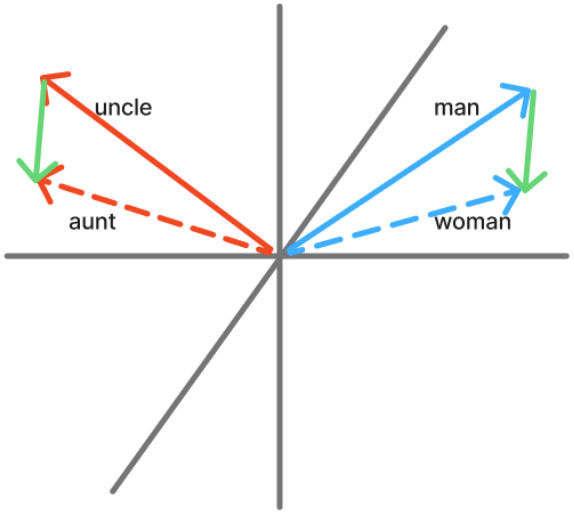


Each word is represented by a vector in this 3D space. The three axes x,y, and z each represent different qualities of the words. For instance, these qualities could include being an adjective, an action, or a pronoun.

Embedding Dimensions

- Here is another example of how directions in the embedding space can have meanings:

Let's say we have vectors for "man" and "woman".



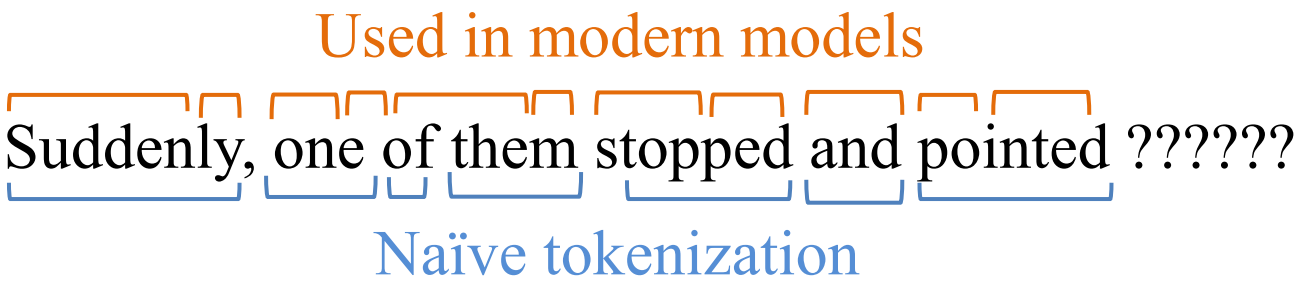
- If we take the difference between these two vectors, we get another vector z. In other words, if we add vector z to the embedding of 'man', we get the embedding of 'woman'.

- Similarly, if we add the vector z to the embedding of 'uncle', we might get the word 'aunt'!

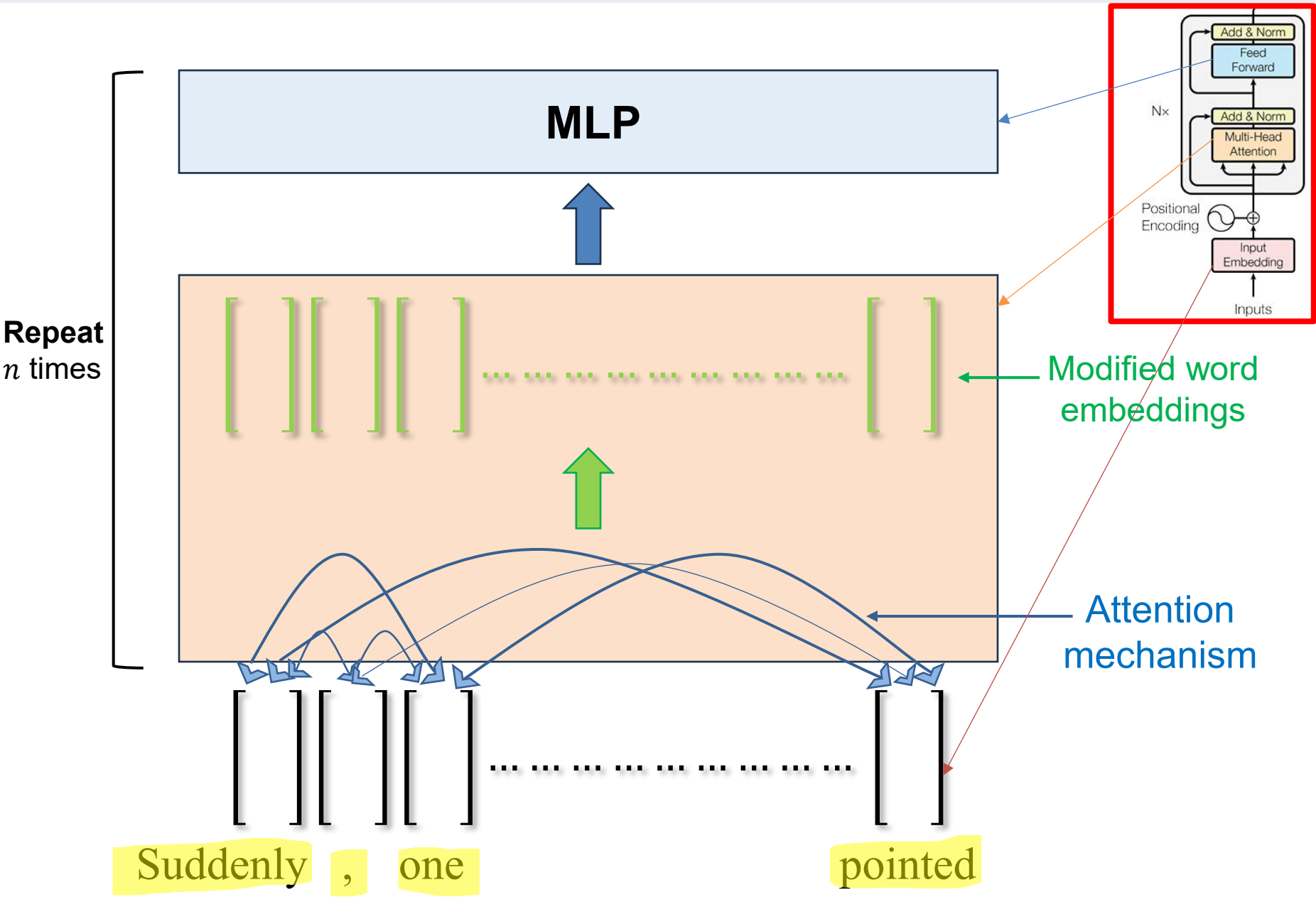
This means that the geometric co-alignment between 'man' and 'woman', i.e. that direction in the embedding space, "understands" the notion of gender!

Note on Tokens and Embeddings

- In reality, embeddings are typically very high-dimensional. For example, GPT-2 used 768-dimensional word embeddings.
  - However, the intuition remains the same.
  - This helps the model encode more complex qualities of a token, such as gender, plurality, hierarchy, appearance, belonging, etc, which improves the model "understanding".
- In reality, for applications like text generation, tokenization does not use whole words, but instead parts of words, i.e. groupings of characters that are commonly seen together. There are many research papers on this, but for simplicity in this course we will treat a token as one word.

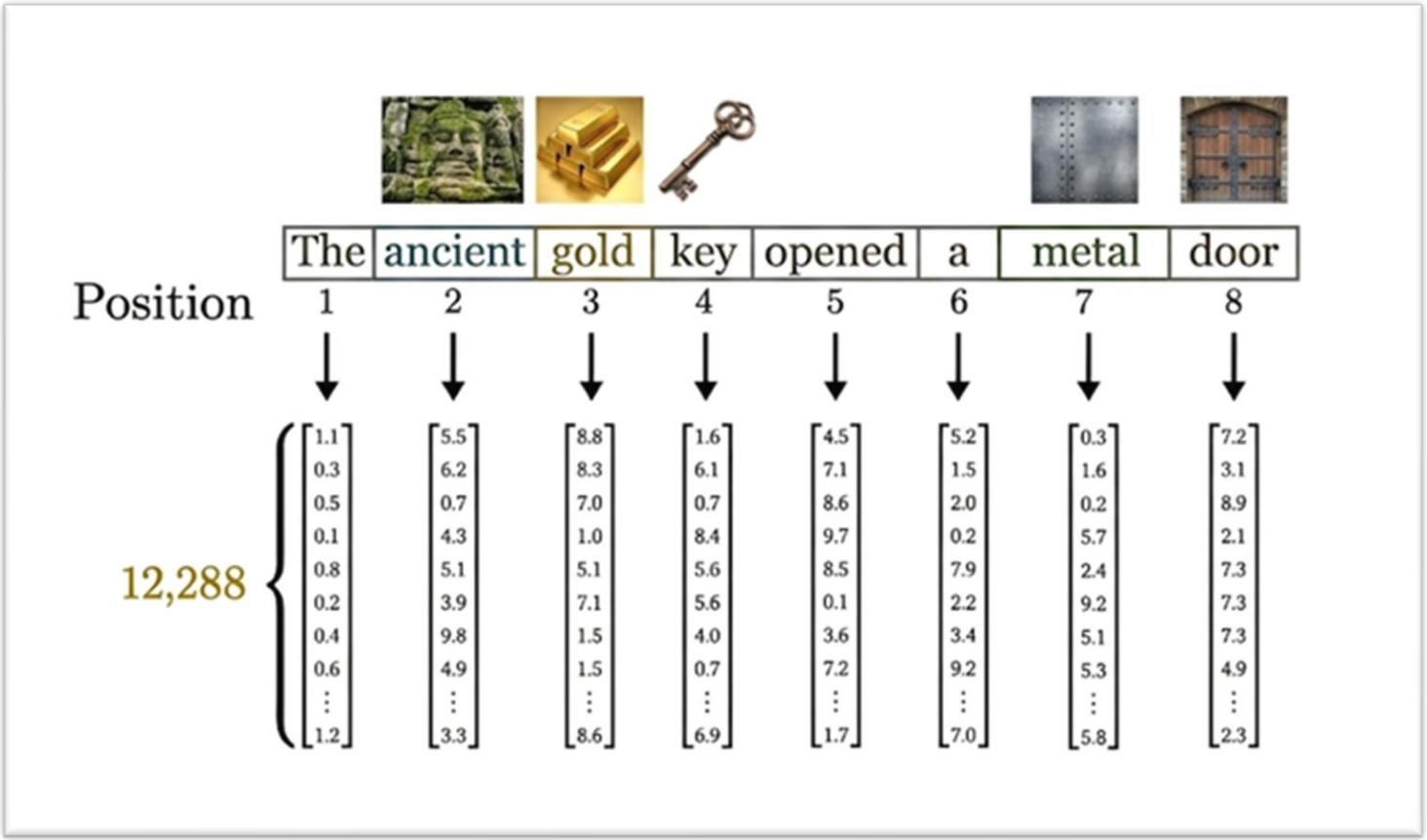


Simplified Architecture of an Encoder



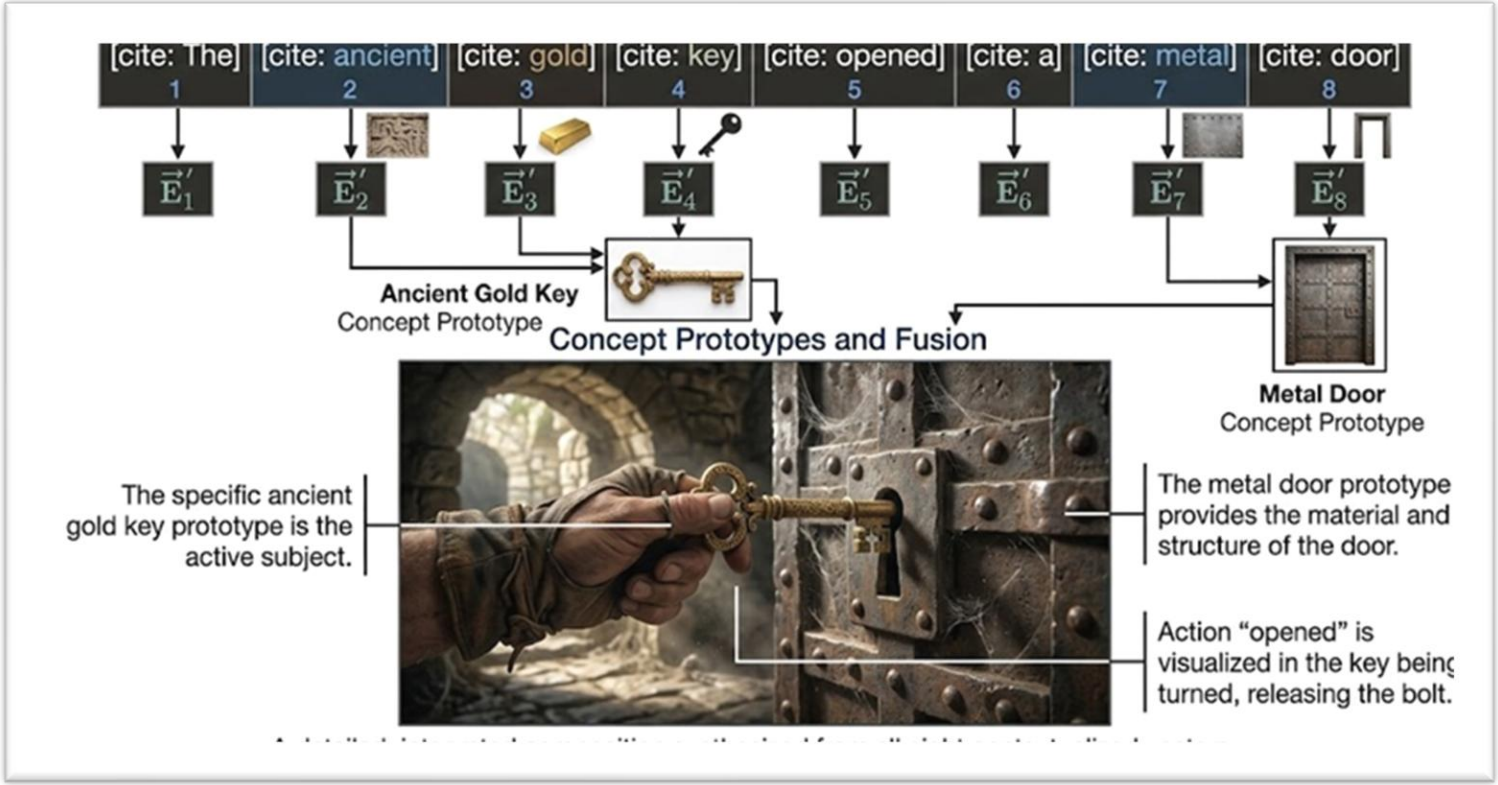
Core Concept of Attention

- Consider this example of a sentence, using 12,288-dimensional embeddings. For the sake of this example, let's assume the positional information is somehow included in the embeddings.



Core Concept of Attention

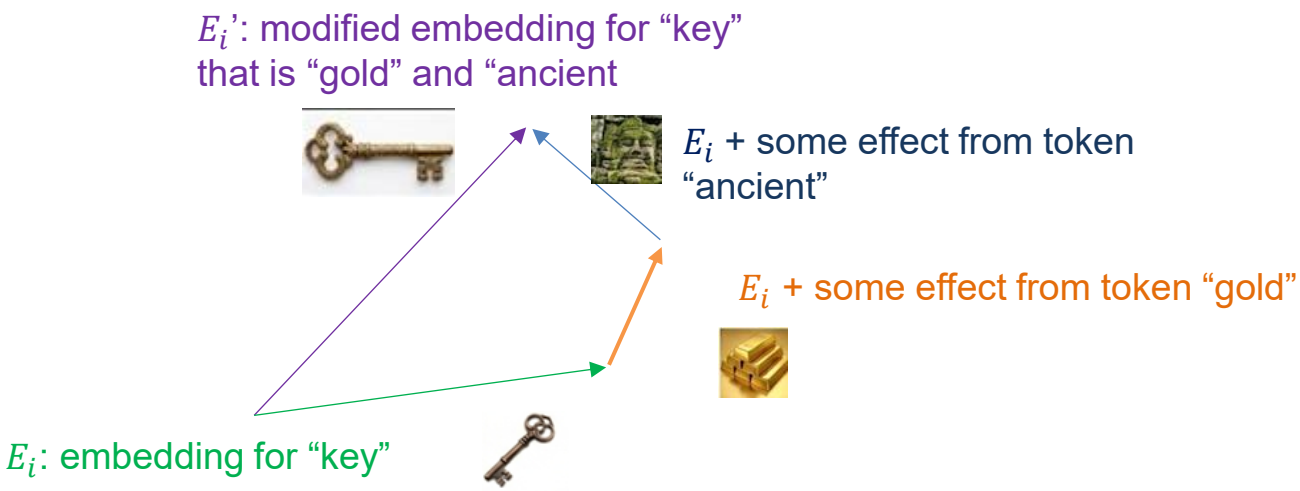
- The word “key” receives context from all the other words, but mainly “ancient” and “gold”, which changes the embedding vector from $\vec{E}_4$ to $\vec{E}_4{}'$.
- Likewise, the word “door” changes to $\vec{E}_8{}'$.



You can imagine that **changing** the embedding vector is the like **moving** the vectors in D-dimensional space closer to their true meanings in the given context.

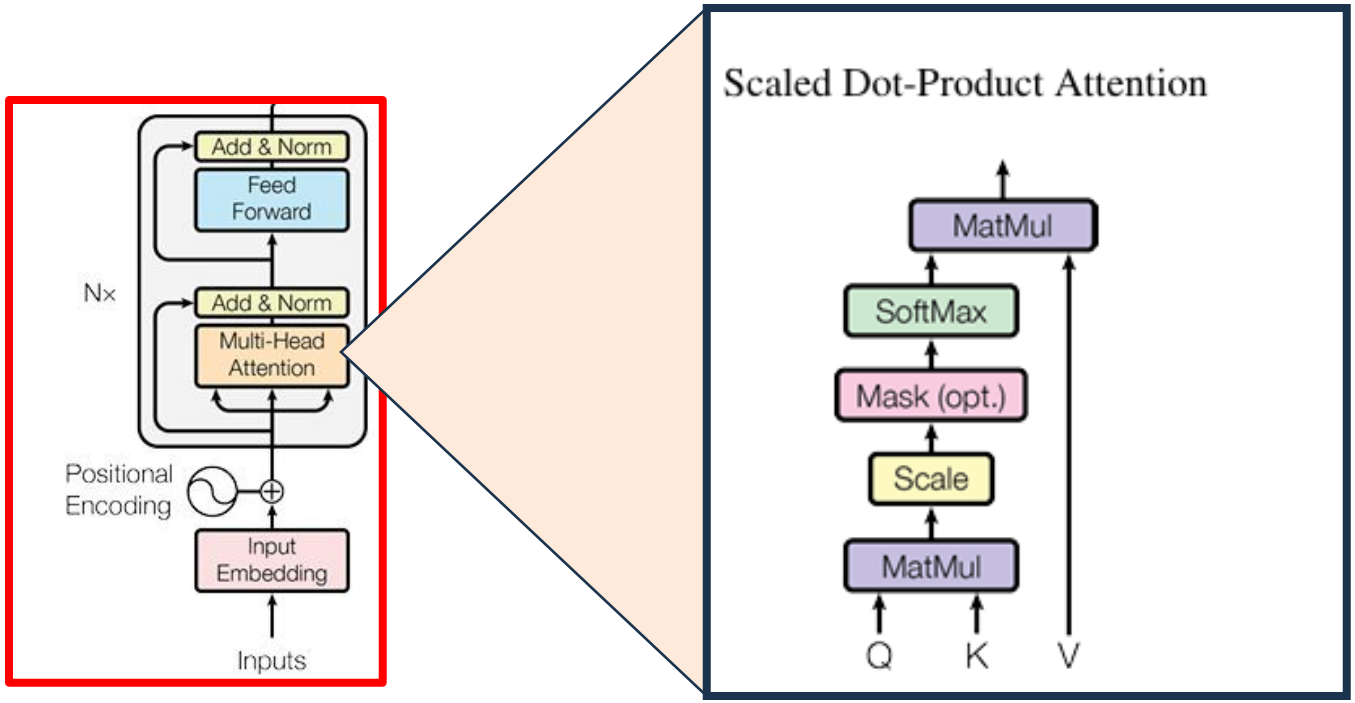
Geometric View

- In a D-dimensional space, this can be viewed as follows:



Attention in Transformers

- Let's look at the attention mechanism within transformers, and how it works in detail.



- Ignore the "multi" as we will explain "single-head attention" first
- We will first explain Queries, Keys, and Values
  - Queries (Q): What I'm looking for
  - Keys (K): What I can offer
  - Values (V): What value I will give you if you choose me

Queries, Keys and Values Intuition

Consider how YouTube works when you search:

Scenario: You search for *"funny cat videos"*

Component	Meaning	Example
Query (Q)	What you're searching for	*"funny cat videos"*
Key (K)	Each video's tags/metadata	*"cats, humor, animals"*
Value (V)	The actual video content	The video itself

Matching Query with Keys Attention score = how well your query matches each video's key → YouTube ranks videos by relevance (this is the dot product $Q \cdot K^T$). A video on “cats fighting" gets a high score, while a video on "Deep Sea Creatures" gets a low score.

Output = weighted sum of values → your feed is a blend dominated by the most relevant videos

Queries

- Consider the sentence is *"The bank by the river was steep"*
- Each token has an embedding **E**. Now the token **"bank"** is ambiguous — does it mean a financial bank or a riverbank?
- *"bank" sends out a query:* **"What kind of thing am I?"**
- This query vector is computed as: $Q = E_{bank} \cdot W_Q$
- **"bank"** is essentially **raising its hand and asking a question to all other tokens.**

Token	What "bank" is asking toward
"river"	✅ High score → *"I'm probably a riverbank"*
"steep"	✅ Medium score → reinforces physical terrain
"The"	❌ Low score → not informative

- The **query doesnot know the answer** — it just broadcasts a direction in vector space, and whatever keys align with that direction respond strongly.

Keys

- The next concept is that of keys, which are generated by every token in the input.
- The keys are also computed by multiplying the original input embedding with the key matrix as: $K = E \cdot W_K$
- The key vector is each token **advertising itself,** essentially answering: *"What am I? What information do I offer?"*
- Example: *"The bank by the river was steep"*
  - **"river"** has a key that advertises → *"I am a natural landmark, geographic context"*
  - **"steep"** has a key that advertises → *"I describe a physical incline"*
  - **"the"** has a key that advertises → *"I am a determiner, low semantic content"*

Computing Attention

- Excerpt from “Attention is All You Need” paper by Vaswani et. al 2017 on method of computing attention using the key and query vectors:

The two most commonly used attention functions are additive attention [2], and dot-product (multiplicative) attention. Dot-product attention is identical to our algorithm, except for the scaling factor of $\frac{1}{\sqrt{d_k}}$. Additive attention computes the compatibility function using a feed-forward network with a single hidden layer. While the two are similar in theoretical complexity, dot-product attention is much faster and more space-efficient in practice, since it can be implemented using highly optimized matrix multiplication code.

(more on the scaling factor later)

Attention Matrix

We take the **dot product** of the Key and Query vectors, for each token, with every other token

The → E_1 $\xrightarrow{W_Q}$ Q_1

ancient → E_2 $\xrightarrow{W_Q}$ Q_2

gold ... ...

door → E_7 $\xrightarrow{W_Q}$ Q_7

The → E_1 $\xrightarrow{W_K}$ K_1

ancient → E_2 $\xrightarrow{W_K}$ K_2

gold

⋮

door → E_7 $\xrightarrow{W_K}$ K_7

$Q_1K_1^T$	$Q_2K_1^T$	...	...	$Q_7K_1^T$
$Q_1K_2^T$	$Q_2K_2^T$	...	...	$Q_7K_2^T$
⋮				⋮
⋮				⋮
$Q_1K_7^T$	$Q_2K_7^T$	...	...	$Q_7K_7^T$

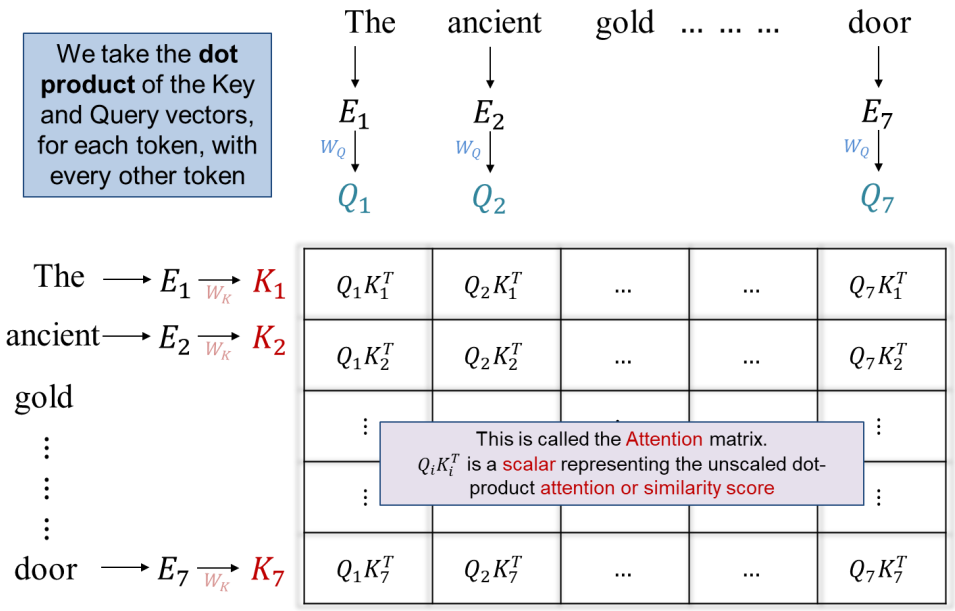
This is called the Attention matrix.
$Q_iK_i^T$ is a scalar representing the unscaled dot-product attention or similarity score

Attention Score

- So, we measure the similarity between two tokens using the dot-product.
- However, there is one issue with the resulting attention matrix:
  - The resulting vectors in the attention matrix, i.e. the columns of the attention matrix are **not normalized.**
  - **Solution:** use the **Softmax** function, applied over each column of the attention matrix individually. This creates a distribution, emphasizes larger scores and suppresses lower scores, while also dealing with negative attention scores.

$$\sigma(t) = \frac{e^t}{\sum e^t}$$

Notation

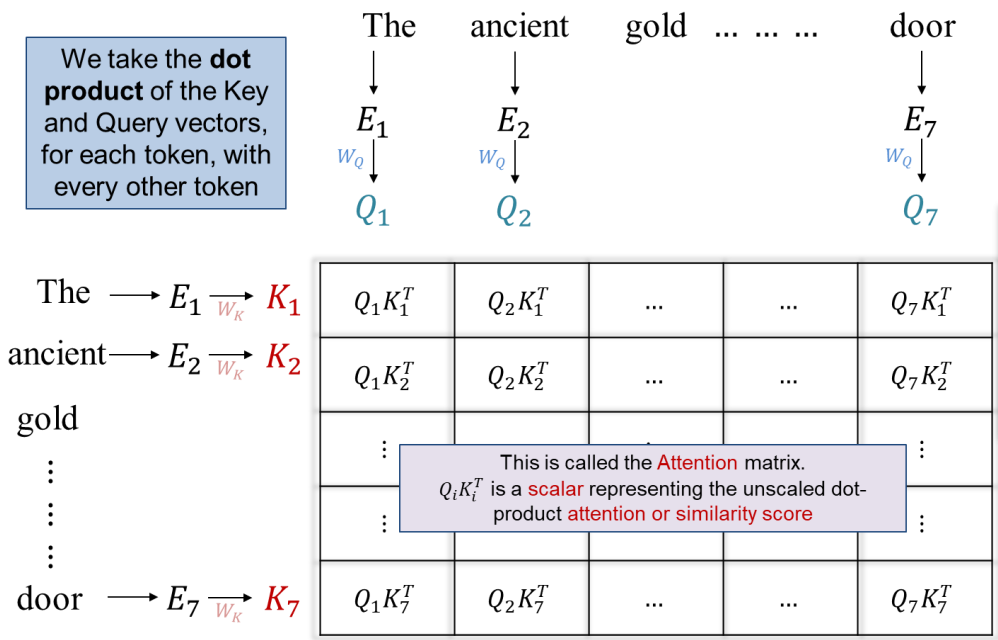


- Input:
  - Input embeddings (one token): E_i (shape = $1 \times d_{model}$)
  - Input embeddings (sequence): E (shape = $\mathrm{n} \times d_{model}$)
  - Query matrix: W_Q (shape = $d_{model} \times d_k$)
  - Key matrix: W_K (shape = $d_{model} \times d_k$)

d_k = dimensionality of the key and query vectors.
In single head attention, $d_k = d_{model}$

Notation

We take the **dot product** of the Key and Query vectors, for each token, with every other token



- Input embeddings (one token): E_i (shape = $1 \times d_{model}$)
- Input embeddings (sequence): E (shape = $n \times d_{model}$)
- Query matrix: W_Q (shape = $d_{model} \times d_k$)
- Key matrix: W_K (shape = $d_{model} \times d_k$)

- What we calculate (assuming n tokens):
  - Query vector (one vector): $Q_i = E_i W_Q$ (shape = $1 \times d_k$)
  - Query vectors: $Q = EW_Q$ (shape = $n \times d_k$)
  - Key vectors: $\mathrm{K} = EW_K$ (shape = $n \times d_k$)
  - Similarities: $S = \frac{1}{\sqrt{d_k}} QK^T$ (shape = $n \times n$)
  - Attention scores: $S' = softmax(S)$ (shape = $n \times n$)

d_k = dimensionality of the key and query vectors. n = the number of input tokens

In single head attention, $d_k = d_{model}$

Scaled Dot-Product Attention

- In the original paper by Vaswani et. Al, we see this formula:

$$\text{Attention}(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

We will talk about the V matrix in a few slides

- Why divide by $\sqrt{d_k}$?
  - Large similarities can cause softmax to saturate, resulting in small (vanishing) gradients.
  - For example, if Q and K are random vectors with mean 0 and variance 1, then the variance of QK^T is directly proportional to the length of the vectors d_k.
    - In other words, for large d_k, the unscaled dot product can be very large
  - Therefore, dividing by $\sqrt{d_k}$ helps stabilize the gradients and improves training.
  - Hence, we call it the **scaled** dot-product attention.

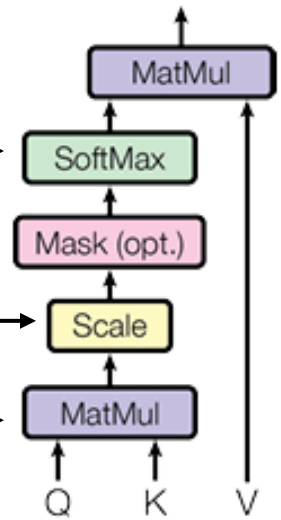
Scaled Dot-Product Attention

Scaled Dot-Product Attention

$$S' = softmax(S)$$

$$S = \frac{QK^T}{\sqrt{d_k}}$$

$$QK^T$$



"Mask" means we hide future tokens (no look ahead) – this is only applied in the **decoder** (more on this later)

This is now the last remaining step, we multiply S' by V

Recap: Attention Matrix So Far

The main idea for what we have done so far is to identify which tokens from the sentence to "**attend to**" the other tokens, and make a distribution out of the resulting attention scores

The ancient gold ... ... door

$$\text{The} \rightarrow E_1 \xrightarrow{W_Q} Q_1 \qquad \text{ancient} \rightarrow E_2 \xrightarrow{W_Q} Q_2 \qquad \text{door} \rightarrow E_7 \xrightarrow{W_Q} Q_7$$

$$\text{The} \rightarrow E_1 \xrightarrow{W_K} K_1$$

$$\text{ancient} \rightarrow E_2 \xrightarrow{W_K} K_2$$

$$\text{gold} \rightarrow E_3 \xrightarrow{W_K} K_3$$

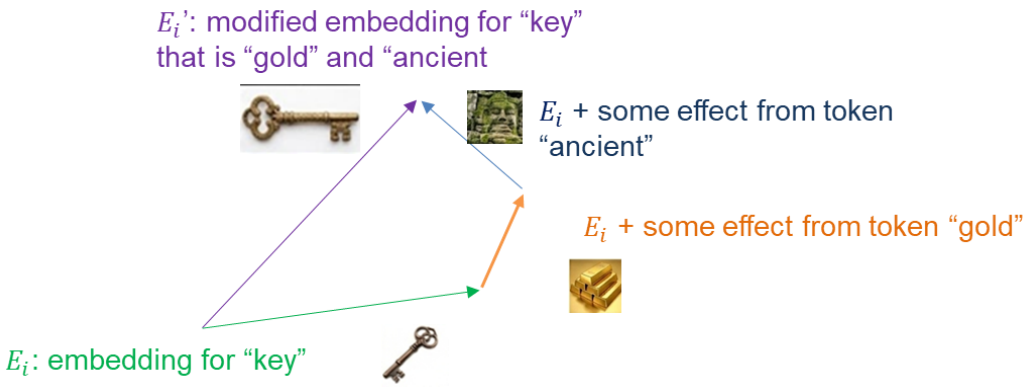
$$\vdots$$

$$\text{door} \rightarrow E_7 \xrightarrow{W_K} K_7$$

⋮	0.03	...	...	⋮
⋮	0.46	...	...	⋮
⋮	0.41	⋱		⋮
⋮	⋮		⋱	⋮
⋮	0	...	...	⋮

How We Use These Scores: Values (V)

- Now that we have normalized attention scores, we need to identify the vector that will move the current embedding vectors E_i to their new positions E'_i

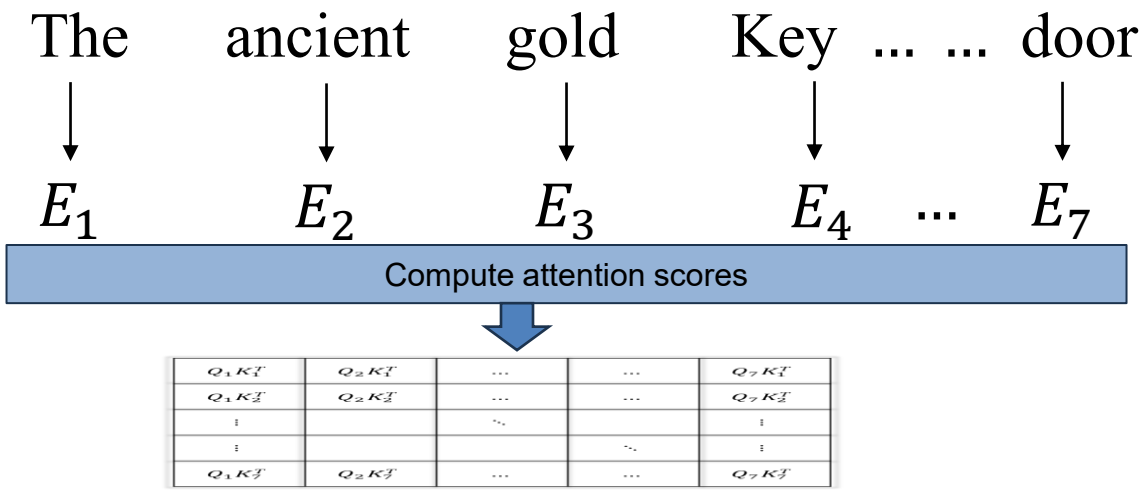


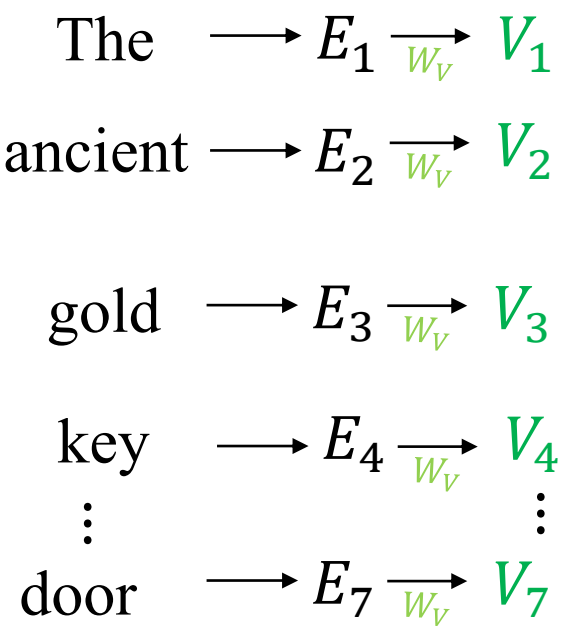
- This is done using the **Value** vectors V_i for each token. The notation is very similar to the Query and Key matrices/vectors:
  - Value matrix: W_V (shape = $d_{model} \times d_v$)
    - **Note that the second dimension d_v can differ from d_k that we saw for Q and K**
  - Value vectors: $V = EW_V$ (shape = $\mathrm{n} \times d_v$)
- In the original *Attention is All You Need* paper, the author set:

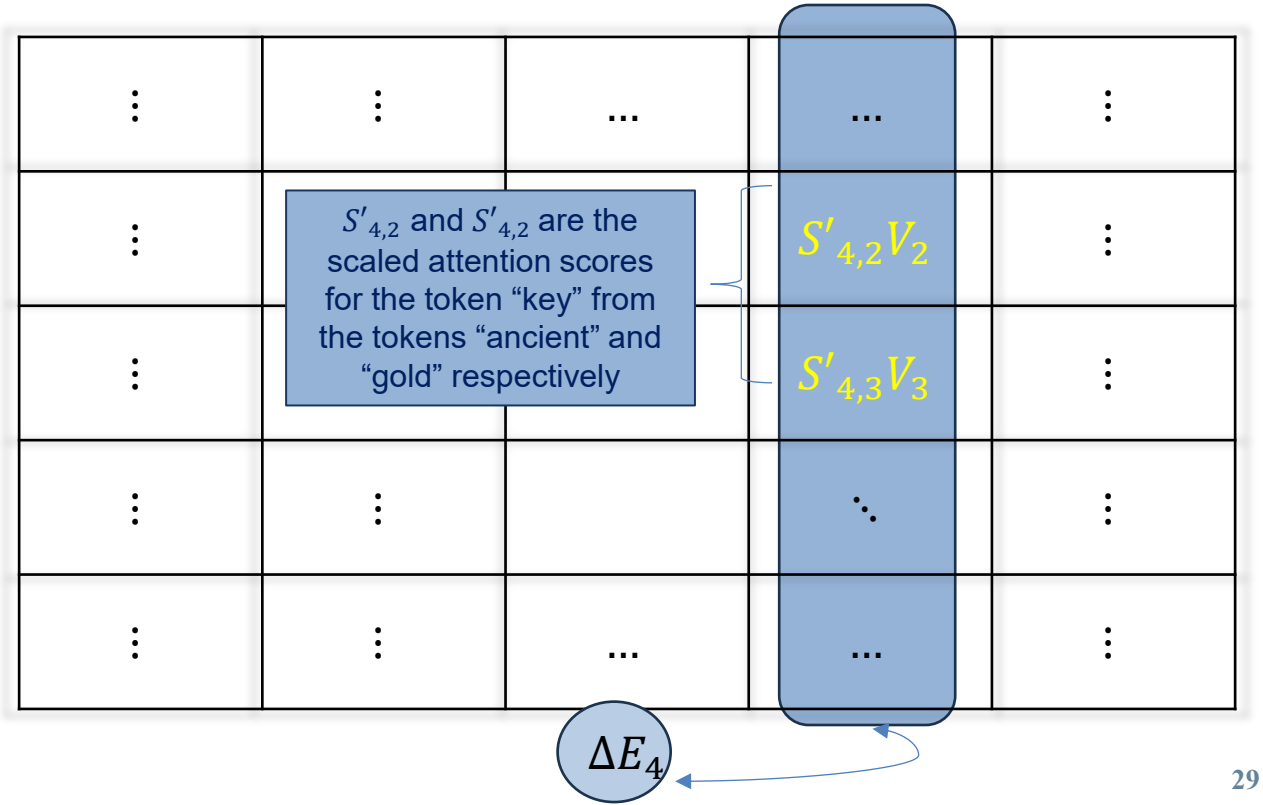
$$d_k = d_v = d_{model} \text{ (in case of single head)}$$

Attention Matrix: Final

The columns of the attention matrix are the dot-product of S'_i with V. The columns now represent vectors ΔE_i that we can **add** to the original embeddings to change their position according to the context

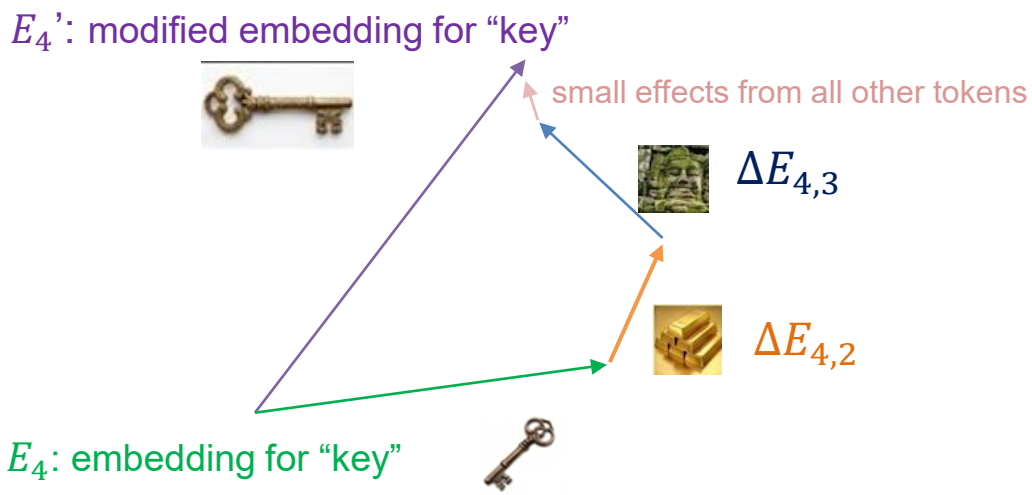






Moving the Original Embeddings

- These vectors ΔE_i represent the "effect" that the other tokens have on token i. Adding these ΔE_i's to the original E_i gives us the modified embeddings.

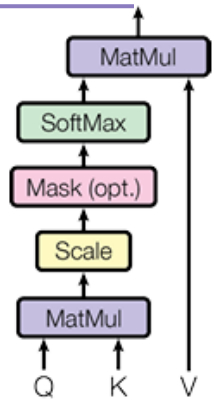


Note: this is only a 2D representation of the "modifications"; in reality, each of these "modifications" is happening along different dimensions of the embedding vector

- We do this for all input tokens:

$$
\begin{array}{ccccc}
E_1 & E_2 & E_3 & \dots & E_n \\
+ & + & + & & + \\
\Delta E_1 & \Delta E_2 & \Delta E_3 & \dots & \Delta E_n \\
\downarrow & \downarrow & \downarrow & & \downarrow \\
E_1' & E_2' & E_3' & \dots & E_n'
\end{array}
$$

Scaled Dot-Product Attention

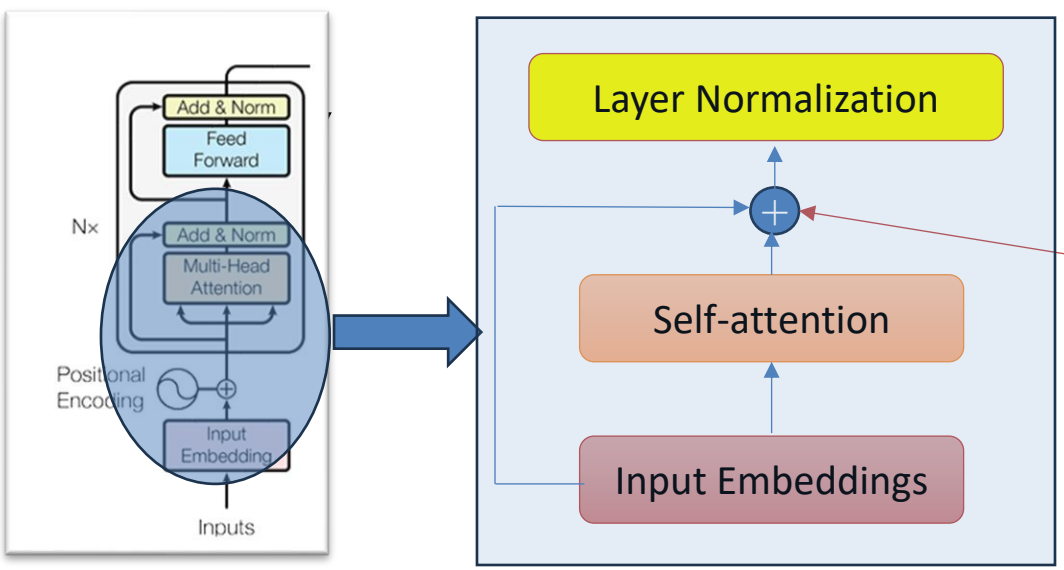


Single Head Attention

- This entire process is called **single head** attention.

E_1	E_2	E_3	…	E_n
$+$	$+$	$+$		$+$
ΔE_1	ΔE_2	ΔE_3	…	ΔE_n
$\downarrow$	$\downarrow$	$\downarrow$		$\downarrow$
E_1'	E_2'	E_3'	…	E_n'

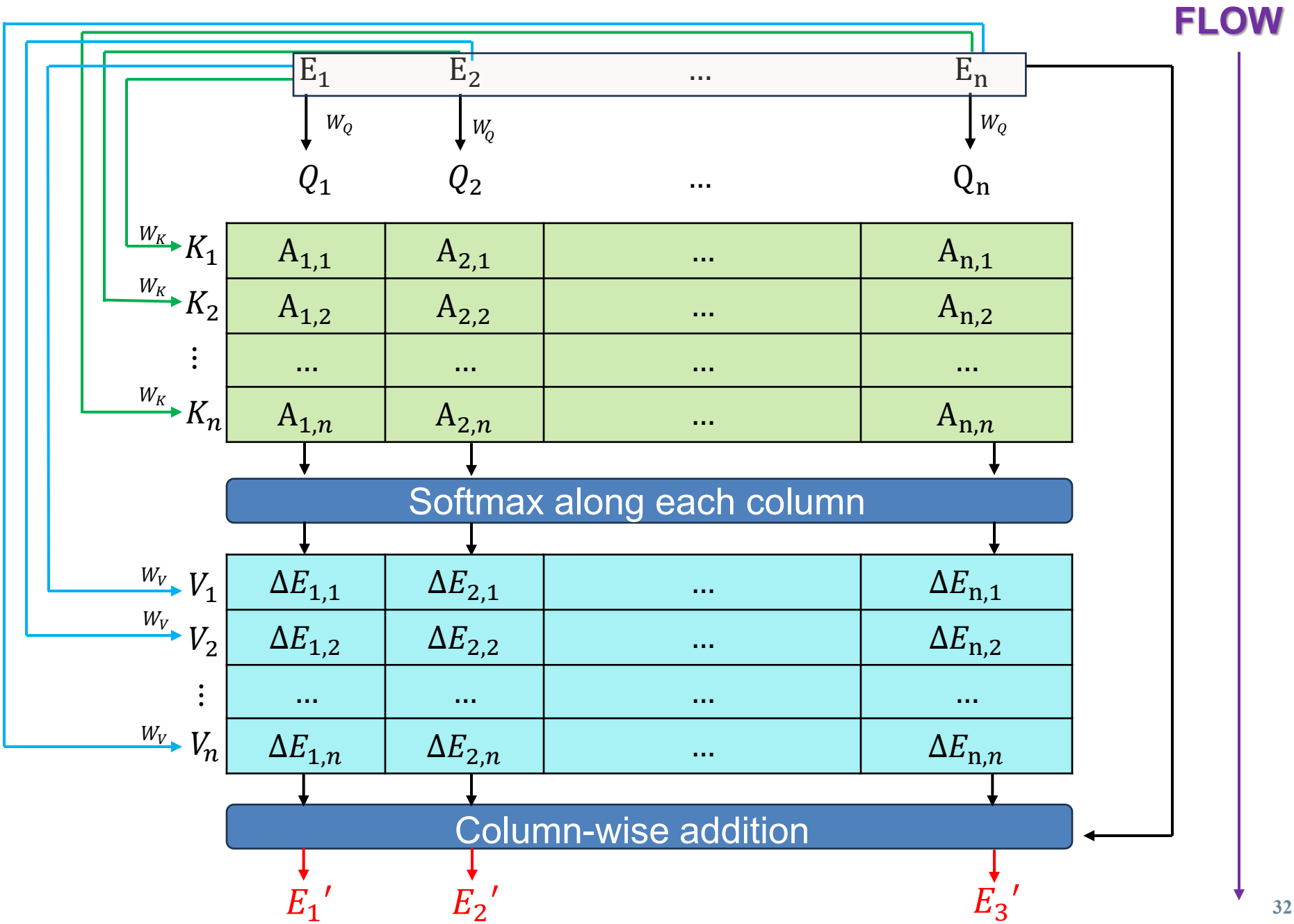
- More specifically, this is called single head **self-attention**, which is different from cross-attention (more on this later).



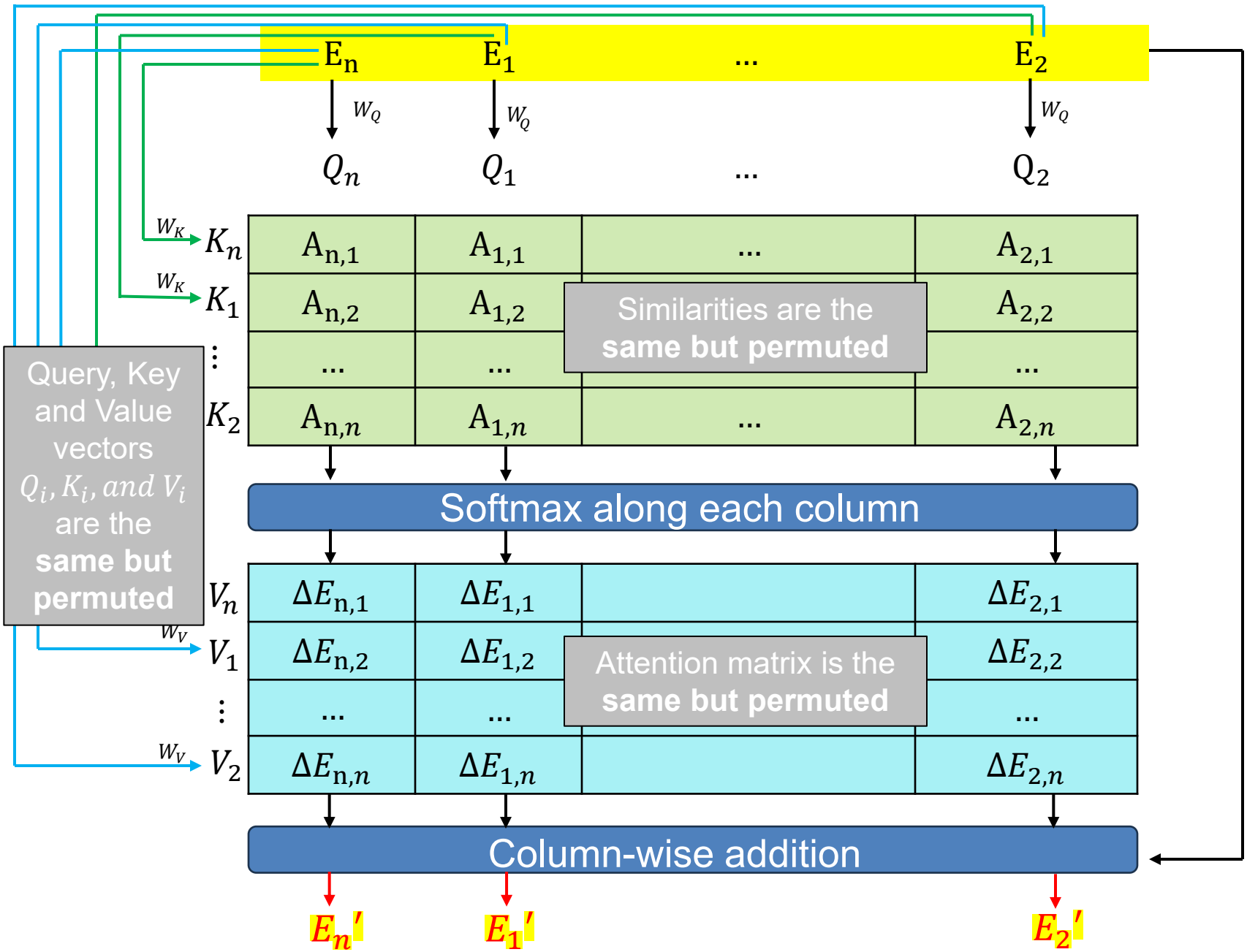
We will talk about the 'multi' in "Multi-Head Attention, and the 'norm' in "Add & Norm" in a few slides.

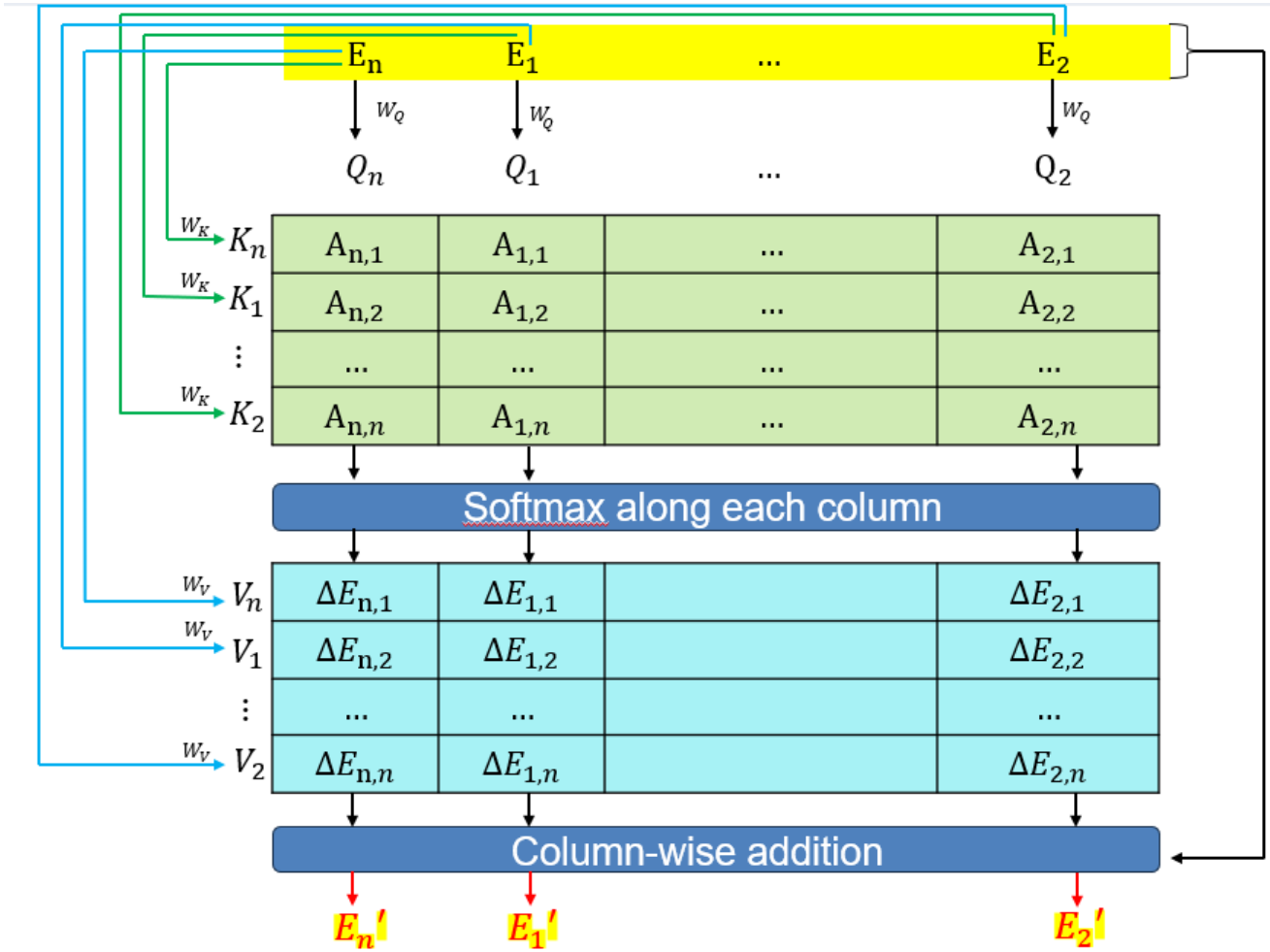
This addition also serves as a "residual connection", which helps with gradient flow.

Overall Architecture So Far



What if We Permute the Input Sequence?





Output is the **same but permuted**
→ Self-attention layer is **Permutation Equivariant**
$f(s(x)) = s(f(x))$

Positional Encodings

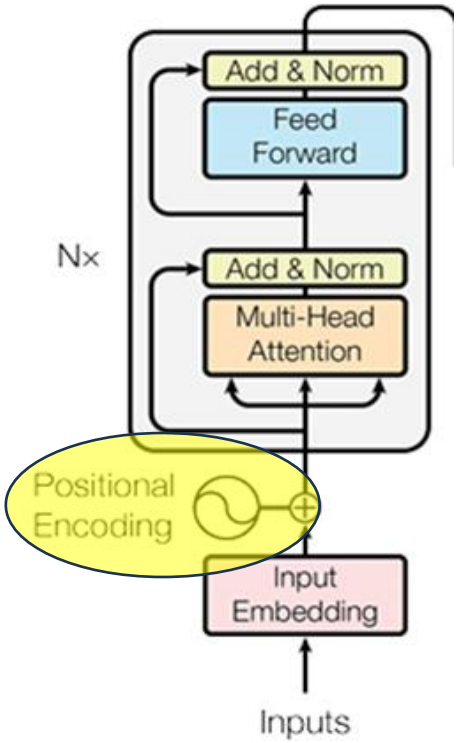
- Permutation equivariance means that self-attention does not care about the order of the input.
- **Solution:** we need to encode some sort of positional information about the relative or absolute position of the tokens, into the embeddings. There are various ways to achieve this, but in the paper:

$$PE_{(pos,2i)} = \sin\left(\frac{\text{pos}}{(10000)^{2i/d_{model}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{\text{pos}}{(10000)^{2i/d_{model}}}\right)$$

  - pos is the position of the token in the sequence
  - i is the dimension index
  - d_{model} is the embedding dimensionality (e.g. 512)
- The intuition is what really matters for self-attention is

how far apart words are (i.e., their relative positions).



chose this function because we hypothesized it would allow the model to easily learn to attend by relative positions, since for any fixed offset k, PE_{pos+k} can be represented as a linear function of PE_{pos}.

Example

- Here is an example with a 4-dimensional embedding ($d_{model} = 4$) and token position (pos) is 3:
  - Index 0: $PE(3,0) = \sin\left(\frac{3}{10000^{\frac{0}{4}}}\right) = \sin(3)$
  - Index 1: $PE(3,1) = \cos\left(\frac{3}{10000^{\frac{0}{4}}}\right) = \cos(3)$
  - Index 2: $PE(3,2) = \sin\left(\frac{3}{10000^{\frac{2}{4}}}\right)$
  - Index 3: $PE(3,3) = \cos\left(\frac{3}{10000^{\frac{2}{4}}}\right)$

$$PE_{(pos,2i)} = \sin\left(\frac{\text{pos}}{10000^{2i/d_{model}}}\right)$$
$$PE_{(pos,2i+1)} = \cos\left(\frac{\text{pos}}{10000^{2i/d_{model}}}\right)$$

$$P_i(t)^T = \begin{bmatrix} \sin(\omega_1, t) \\ \cos(\omega_1, t) \\ \sin(\omega_2, t) \\ \cos(\omega_2, t) \\ \vdots \\ \sin(\omega_{d/2}, t) \\ \cos(\omega_{d/2}, t) \end{bmatrix}^T_{d_{model}\times 1}$$

where $\mathrm{t} = \mathrm{pos}, \mathrm{d} = \mathrm{d_{model}}$, and

$$\omega_i = \frac{1}{10000^{\frac{2i}{d_{model}}}}$$

Input Embeddings

- The positional encodings are added to the token embeddings element-wise before feeding them into the Encoder:

Input to Transformer = Token Embeddings + Positional Encodings

$$\text{Input Embeddings} = E_i + P_i(t)$$

- The embedding vector and the positional encoding vector have the same dimension (d_{model}).
- This means that each E captures its semantic meaning (from the word embeddings) as well as its position in the sequence.

- Learned vs. Sinusoidal Positional Encodings:
  - The original Transformer paper uses sinusoidal positional encodings, which are fixed and not learned during training.
  - Some modern Transformers use learned positional embeddings, which allow the model to learn position representations dynamically but may not generalize as well to longer sequences.
  - In the original paper. The authors say “We also experimented with using learned positional embeddings instead, and found that the two versions produced nearly identical results”

Multi Head Attention (MHA)

- Instead of performing a single attention function with d_{model} dimensional queries Q, keys K and values V, the authors of the paper found it that it was better to **project** them down to d_k, d_k and d_v-dimensions, respectively, and to have h different projections. Then, they performed attention using each of these h sets of Q_i, K_i and V_i, **in parallel**.
- Each "set of projections" is called a **head**, and each head has its own learnable projections (weight matrices) for W_i^Q, W_i^K and W_i^V
- Mathematically, assume we have n tokens, d_model is the size of the embeddings, and for each head i, we have:

$$E \in \mathbb{R}^{n \times d_{model}}, W_i^Q \in \mathbb{R}^{d_{model} \times d_k}, W_i^K \in \mathbb{R}^{d_{model} \times d_k}, W_i^V \in \mathbb{R}^{d_{model} \times d_v}$$

$\therefore Q_i = EW_i^Q \in \mathbb{R}^{n \times d_k}$ and similarly $K_i \in \mathbb{R}^{n \times d_k}$, and $V_i \in \mathbb{R}^{n \times d_v}$

In Single-Head Attention:
$d_k = d_{model}$

In Multi-Head Attention (MHA):
$d_k = \frac{d_{model}}{h}$

Multi Head Attention

- Each of those heads can be thought of as regular single-head attention as before, but since the computation between each head is **independent**, we can do the computations in **parallel** which is more efficient.
- After all the computations, we have h different $n \times d_v$ dimensional outputs (one per head). In the paper, they concatenate all h matrices into one $n \times (h \cdot d_v)$-dimensional matrix, and then do another linear projection to map it back down to $n \times d_{model}$

$$\mathrm{MultiHead}(Q, K, V) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h) W^O$$
$$\text{where } \mathrm{head}_i = \mathrm{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

$$W^O \in \mathbb{R}^{hd_v \times d_{model}}$$

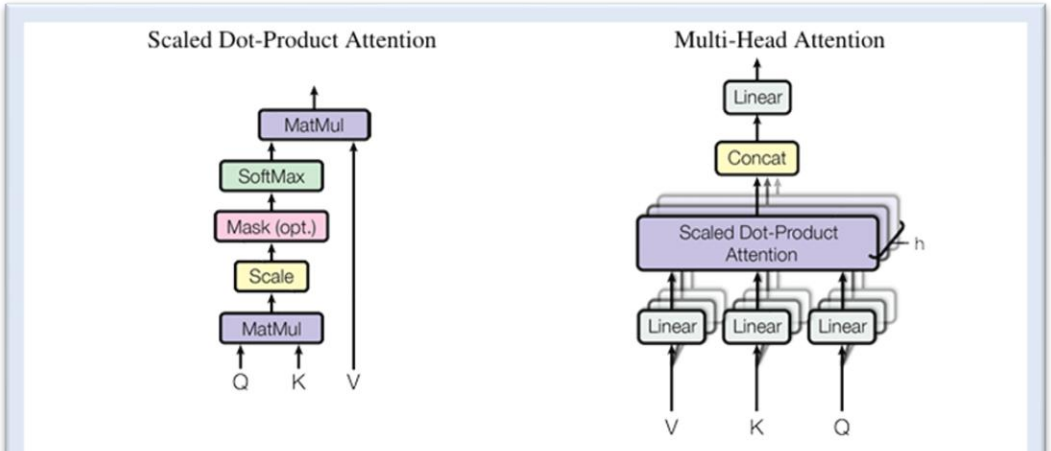


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

Example

Here is an example to demonstrate the dimensionality of the various matrices in single-head attention vs multi-head attention.

- Consider: $d_{model} = 512$ and for multi-head, we'll use $h = 8$ with $d_k = d_v$

Single-head:

$$Q = E_{n\times 512} W^{Q}_{512\times 512}$$

$$K = E_{n\times 512} W^{K}_{512\times 512}$$

$$V = E_{n\times 512} W^{V}_{512\times 512}$$

Output is $n \times 512$

Multi-head:

$$d_k = d_v = \frac{d_{model}}{h} = \frac{512}{8} = 64$$

$$Q_i = E_{n\times 512}\,(W_i)^{Q}_{512\times 64}$$

$$K_i = E_{n\times 512}(W_i)^{K}_{512\times 64}$$

$$V_i = E_{n\times 512}(W_i)^{V}_{512\times 64}$$

for $i = 1,2, \ldots 8$, and

$W^{O} \in \mathbb{R}^{512\times 512}$ (output linear project.)

Output is also $n \times 512$

Why Multi-Head Attention

- While computation in theory is similar (multi-head attention has an extra linear projection), however, with parallel computing, the multi-head attention is more efficient.
- Multiple representations: each head learns and focuses on different contexts between its Q and K matrices. Multiple heads allows for capturing diverse patterns and relationships between tokens.
- Multi-head attention is better at handling long-range dependencies between tokens.
- Multi-head attention is like an **ensemble** of single-head attention heads working independently
  - Leads to being more robust against noise, and less prone to overfitting to relationships in the training tokens.

Multihead Attention Example with Code

Let's use input sequence = "Cat chases mouse around"

- Let's say we have batch size of 1 (model only processes one sequence at a time), and we use 16 heads, with a d_{model} of 512.
- So, we know $d_k = \frac{512}{16} = 32$

Step 1) Convert input sequence to correct shape.

- Input X should have shape (batch_size, seq_len, d_model)
  - Here, that is shape (1, 4, 512)

Example 4 token input with $d_{model} = 16$

```
tensor([[[0.8740, 0.5367, 0.7634, 0.1335, 0.3225, 0.7254, 0.5963, 0.7987,
          0.1375, 0.3413, 0.2120, 0.6607, 0.9538, 0.2828, 0.8572, 0.5809],
         [0.6045, 0.3031, 0.2739, 0.6120, 0.9970, 0.6298, 0.2231, 0.4907,
          0.0724, 0.2328, 0.9151, 0.0637, 0.9006, 0.8245, 0.9452, 0.1708],
         [0.6710, 0.0528, 0.5031, 0.6287, 0.0356, 0.6554, 0.9312, 0.8375,
          0.1364, 0.5978, 0.3612, 0.2856, 0.8028, 0.7691, 0.6069, 0.7731],
         [0.9179, 0.2706, 0.1264, 0.8726, 0.9410, 0.5357, 0.8354, 0.1887,
          0.7399, 0.7123, 0.4765, 0.8105, 0.3125, 0.3201, 0.2266, 0.9239]]])
```

```
batch_size = 1
seq_len = 4
d_model = 512
num_heads = 16
d_k = d_model // num_heads  # Dimension per head (32)

# Simulated input embedding
E = torch.rand(batch_size, seq_len, d_model) #random numbers generated from a uniform distribution [0, 1)
print("Input shape:", E.shape)  # Expected: (1, 4, 512)
print("Input embeddings:", E)
```

$$Q = E\,W^Q, \qquad K = E\,W^K, \qquad V = E\,W^V$$

Step 2) Linear projection to Q, K, V

- Query vector (one vector): $Q_i = E_i W_Q$ (shape = $1 \times d_k$)
- Query vectors: $Q = EW_Q$ (shape = $n \times d_k$)
- Key vectors: $K = EW_K$ (shape = $n \times d_k$)

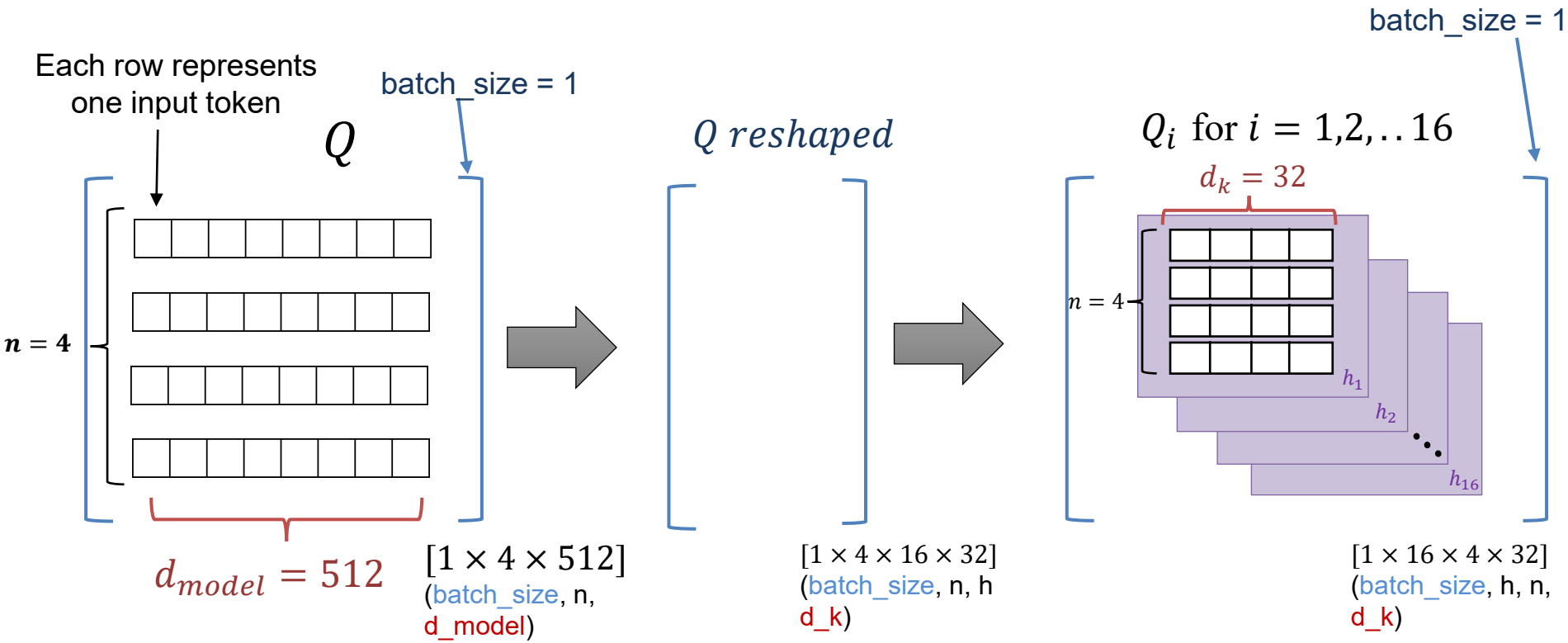
```
W_Q = torch.nn.Linear(d_model, d_model) # (512, 512)
W_K = torch.nn.Linear(d_model, d_model) # (512, 512)
W_V = torch.nn.Linear(d_model, d_model) # (512, 512)

# Generate Q, K, V matrices
Q = W_Q(E) # (1, 4, 512)
K = W_K(E) # (1, 4, 512)
V = W_V(E) # (1, 4, 512)

print("Q:", Q)
```

torch.nn.Linear creates a linear projection layer; it is a dense, fully-connected layer connecting the input E and the respective output

- Initialize learnable matrices of dimension 512x512, for W^Q, W^K, W^V, which will be randomly initialized
- Apply linear layer to input X, using Q = W_Q(E), which computes $Q = E\,W^Q$

Step 3) Split into multiple heads: $Q, K, V \rightarrow Q_i, K_i, V_i, i = 1,2, \ldots 16$

- First, we **reshape** (batch_size, seq_len, d_model) into shape (batch_size, seq_len, h, d_k)
- Then, we **permute** this into shape (batch_size, h, seq_len, d_k)



```python
# Reshape and permute for multi-head attention
Q = Q.view(batch_size, seq_len, num_heads, d_k).permute(0, 2, 1, 3)
K = K.view(batch_size, seq_len, num_heads, d_k).permute(0, 2, 1, 3)
V = V.view(batch_size, seq_len, num_heads, d_k).permute(0, 2, 1, 3)

print("Shape of Q after splitting into heads:", Q.shape)  # (1, 16, 4, 32)
print("Shape of K after splitting into heads:", K.shape)  # (1, 16, 4, 32)
print("Shape of V after splitting into heads:", V.shape)  # (1, 16, 4, 32)
```

Step 4) Scaled dot-product attention on each head

$$\text{Attention}(Q_i, K_i, V_i) = softmax\left(\frac{Q_i K_i^T}{\sqrt{d_k}}\right) V_i$$

```
import torch.nn.functional as F
# Calculate attention scores
scores = torch.matmul(Q, K.transpose(-2, -1)) / (d_k ** 0.5)
print("Attention scores shape:", scores.shape)  # (1, 16, 4, 4)
# Apply softmax to get attention weights
attention_weights = F.softmax(scores, dim=-1)
print("Attention weights shape:", attention_weights.shape)  # (1, 16, 4, 4)
# Calculate the weighted sum of values
attention_output = torch.matmul(attention_weights, V)
print("Attention output shape:", attention_output.shape)  # (1, 16, 4, 32)
```

Transpose is done on last two dimensions

Softmax along columns

Step 5) Concatenate heads and final linear layer

```
# Concatenate attention output from all heads
# (batch_size, seq_len, num_heads, d_k) -> (batch_size, seq_len, d_model)
attention_output = attention_output.permute(0, 2, 1, 3).contiguous()
attention_output = attention_output.view(batch_size, seq_len, d_model)
print("Shape after concatenating heads:", attention_output.shape)  # (1, 4, 512)

# Final linear layer to project back to d_model
W_O = torch.nn.Linear(d_model, d_model)  # (512, 512)
final_output = W_O(attention_output)

print("Final output shape:", final_output.shape)  # (1, 4, 512)
print("Final output:", final_output)  # (1, 4, 512)
```

This is the final ΔE that will be **added** to E to produce E'

```
Shape after concatenating heads: torch.Size([1, 4, 512])
Final output shape: torch.Size([1, 4, 512])
Final output: tensor([[[-0.2283, -0.0407,  0.2618,  ...,  0.2667, -0.1729, -0.0953],
        [-0.2311, -0.0413,  0.2638,  ...,  0.2654, -0.1736, -0.0943],
        [-0.2265, -0.0410,  0.2603,  ...,  0.2658, -0.1742, -0.0931],
        [-0.2282, -0.0397,  0.2624,  ...,  0.2672, -0.1755, -0.0903]]],
```

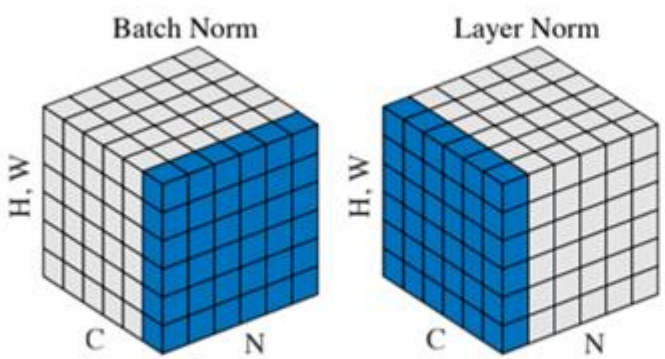
Why We Need The Final Projection

Why not just stop after concatenation?

- The final linear layer blends information using a **weighted combination across heads**, so that different information from each head is taken into consideration
- The final linear weight matrix W^O also consists of **learnable parameters**, meaning that the model can learn complex dynamic relationships between heads and the information that they can capture.
- Of course, if $d_k \neq d_v$ and consequently $h \cdot d_v \neq d_{model}$, then the output after concatenation will not the expected output size, so the final projection ensures that the output of multi-head attention has dimension d_{model}

Layer Normalization

- Recall batch normalization from previous classes.
- We can do similar normalization but instead of normalizing values across a **batch**, we can normalize a **single training** (a single sequence) across the embedding dimension.
  - i.e. the statistics – mean and standard deviation are calculated per vector
- This is called **layer normalization**.
- Layer norm helps with optimization.



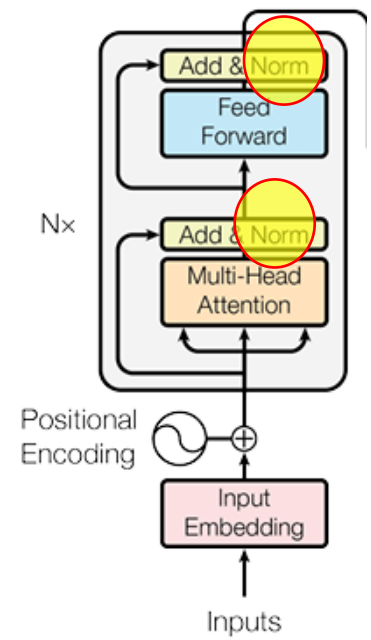
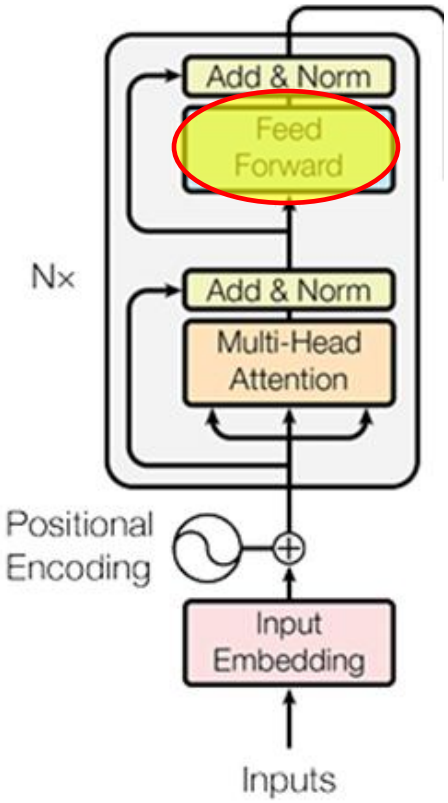


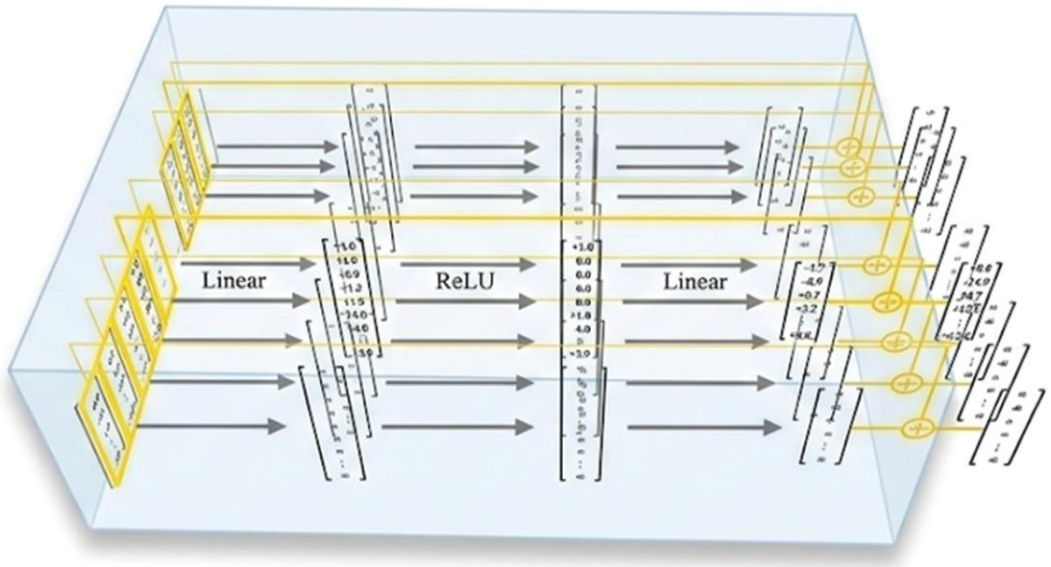
Figure 2. **Normalization methods**. Each subplot shows a feature map tensor, with N as the batch axis, C as the channel axis, and (H, W) as the spatial axes. The pixels in blue are normalized by the same mean and variance, computed by aggregating the values of these pixels.

Feed Forward Network (FFN)

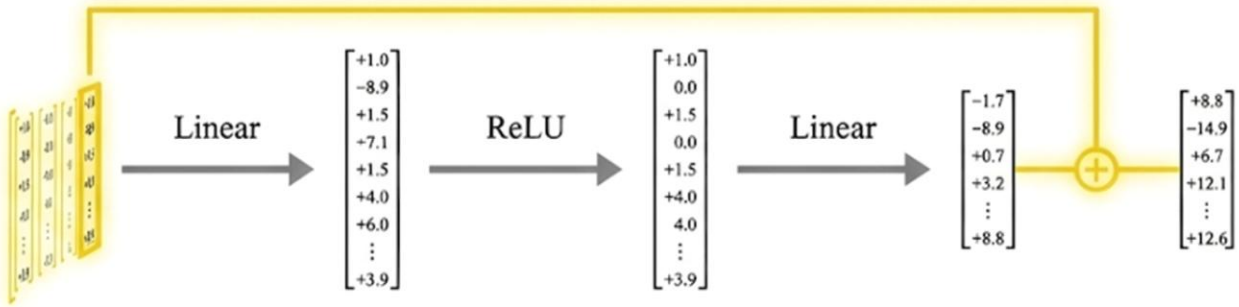
- This is a fully-connected layer, like we studied before (also called an MLP).
- It is applied to each embedding **individually**.
- While the attention mechanism can integrate context, the purpose of this MLP is to introduce **non-linearity**, since all the operations in the attention mechanism are **linear** operations.
  - Non-linearity allows the model to learn even more complex representations beyond what linear attention mechanisms may allow.
- It also acts as a kind of "feature transform"
  - The MLP is usually configured to have **two** linear layers, the first transforms the embeddings to a **higher** dimensionality than d_{model}, the second transforms it back to d_{model}



MLP Applied To Each Vector



MLP



Mathematical Representation of the MLP

First Linear Transformation:

- $X' = ReLU(X\,W1 + b1)$
- X: Input tensor $(batch_size, seq_len, d_model)$
- $W1$: Weight matrix (d_model, d_ff)
- $b1$: Bias vector (d_ff)

- ReLU prevents model from collapsing to a linear transformation

Second Linear Transformation:

- $Y = X'\,W2 + b2$
- $W2$: Weight matrix (d_ff, d_model)
- $b2$: Bias vector (d_model)

- The FFN expands the input dimension from d_{model} to d_{ff}, then projects back to d_{model}.
  - Typically, $d_{ff} = 4 \times d_{model}$. So in the standard transformer, $d_{ff} = 2048$.
  - This “bottleneck” effect helps learn complex interactions.

Next Time: Transformers